Editor Note: Feel free to leave comments and suggestions to make this better for future readers. 😀

Claude Code: 10-Day Onboarding Plan

In this 10-day plan, you'll gradually incorporate Claude Code – Anthropic's AI coding assistant – into your development workflow. Each day builds on the last, introducing new features and best practices. By the end, you should feel comfortable using Claude to navigate your codebase, write and refactor code, run tests, and even integrate with project management tools.

Days 1–2: Setup and First Run

Install and Authenticate Claude Code: Ensure you have Node.js 18+ installed (you likely do; if not, install it first).
Claude Code is distributed in two ways:

1. via NPM – run *npm install -g @anthropic-ai/claude-code* to install the CLI globally. Avoid using sudo with this global installation for security reasons
2. via Bun single-file executable – run *curl -fsSL claude.ai/install.sh | bash*

Once installed, navigate (*cd*) to your project's root directory (the repository you want Claude to work on).

Launch Claude in Your Project: In the project directory, run *claude* to start the CLI. The first launch will prompt you to authenticate with Anthropic – select "Claude account with subscription" and follow the OAuth login steps. Once authenticated, Claude Code will attach to your current project context automatically.

Generate Project Context (CLAUDE.md): Now that Claude is running in your repo, test it out with a simple query. For example, at the *claude>* prompt, ask it to "summarize this project" to see if it can describe your codebase. Next, initialize a memory file for your project by using the */init* command. Type */init* and hit Enter – Claude will generate a *CLAUDE.md* file containing an overview of the project, key commands, and other info. This file serves as a **project memory** that Claude automatically loads every session. Commit the new CLAUDE.md into your repository (you can simply tell Claude "commit the CLAUDE.md to git" and it should stage and commit it for you, or do it manually). *Note:* This file is the AI's

starting memory; you'll customize it later. For now, skim through it and notice sections like build commands or essential files that Claude identified.

Reminder: If your team uses a specific Node version manager or other environment setup, ensure that these are addressed. Claude can handle environment specifics if documented (e.g., if you need `nvm use X` or special shell init, put that info in CLAUDE.md or follow-up instructions). Also, remember Claude is running with your project context, meaning it can read your code and understand the repository structure by default. Always launch it from the project directory so it can recursively find relevant files and the CLAUDE.md memory.

Day 3: First Coding Assistance

With the setup done, try using Claude on a real task. Pick a minor feature or improvement in your codebase that you can implement with AI help. For example, maybe there's a function that needs to be added or a TODO in the code you've been meaning to tackle. Provide Claude with the context and ask it to help:

- **Explore the code:** You can instruct Claude to open or read specific files. For instance, "open the file that handles user login" or "read src/auth.js " – Claude will display the file contents for you in the terminal. This helps it get context beyond the initial memory. Claude can search your repository for relevant references if you ask it to (similar to how a teammate might grep for something).
- **Ask questions:** Don't hesitate to ask Claude questions about the code. You might ask, "How is the logging utility implemented?" or "What does the function *processOrder* do?" Claude will traverse the codebase to answer (it can follow imports, read multiple files, etc.). Anthropic engineers often use Claude in this manner for onboarding to new code, posing questions to it as if it were a colleague.
- **Implement a small change:** Once you and Claude have identified where to make your change, ask Claude to help write the code. For example, "Add a new function to calculate the order total and use it in checkout() ". Claude will draft the code for you. Review the suggestion carefully – ensure it fits your requirements and style. This is a good time to correct it if needed: if the first attempt isn't perfect, clarify requirements or style ("Use our naming convention for functions" or "Don't use external libraries for this"). Claude thrives on specific instructions, so the clearer you are, the better the code it produces.

Work iteratively: read Claude's output, run the code or tests, if possible, to verify it works, and ask Claude to fix any issues. By the end of Day 3, you should have a modest commit (or at least a diff) created largely with Claude's help. It could be a new function, a small feature, or a minor bug – the goal is to get comfortable with Claude reading and writing your code.

Day 4: Debugging and Testing with Claude

Today, use Claude to tackle a bug or failing test in your project. This will get you familiar with using the AI for debugging and ensuring code quality:

- **Identify a target bug:** Perhaps there's a known issue (a failing unit test or a bug report) you can address. If you have failing tests, run your test suite (you can have Claude run it by saying "run the tests"). When tests fail, copy the failing test output or stack trace into Claude's chat. Claude can interpret test failures and often pinpoint the likely cause.
- **Diagnose with Claude:** Ask Claude to help figure out the cause of the bug. For example, "The user profile update test is failing with a null pointer exception – why might that be?". Claude will search through relevant code paths to hypothesize the cause. It might open the file and highlight where a null value could occur (assuming you have the IDE integration configured, which we'll discuss tomorrow). This is similar to how you'd talk through a bug with a teammate – Claude can examine the code faster than a human (at least faster than me), but you should verify its conclusions.
- **Plan and fix:** Once the cause is identified (or at least a theory is formed), ask Claude to outline a fix. If you're confident in the plan, instruct Claude to implement the fix in the code. For example: "Fix the null pointer issue in ProfileService (in file X) by adding a check for undefined email before calling validation." Be explicit about the solution you want, or let Claude propose one and then refine it as needed.
- **Run tests and iterate:** Have Claude rerun the tests to see if the issue is resolved (it can execute the test command as it has shell access). If tests still fail, iterate: ask Claude to read the new error or reconsider its fix. This might take a couple of attempts – a regular part of the process. Claude is quite capable of test-driven development workflows. You can explicitly instruct it to run tests, observe failures, adjust the code, and repeat the process until the tests pass. Guide it with instructions like "only change the implementation, do not alter the test." This ensures it's truly fixing the bug rather than cheating by modifying tests.

Tip: When debugging, encourage Claude to "think" through the problem. You can say, "Think hard about why this error is happening." – Claude has special modes for deeper reasoning if you use triggers like "think harder" or "ultrathink" (these allocate more time for Claude to analyze). This can lead to a more thorough analysis of complex issues.

Day 5: Integrating Claude with Your IDE 💻

At this point, you've been using Claude in the terminal. If you find the terminal interface cumbersome, you can now integrate Claude Code with your preferred IDE, allowing you to work more comfortably. Claude Code supports Visual Studio Code and JetBrains IDEs out of the box:

- **Visual Studio Code:** Open VS Code and your project folder. In VS Code's integrated terminal, run the *claude* command (just as you've been doing). The first time you do this, Claude Code's VS Code extension will auto-install. You should see a Claude panel or button appear. From now on, you can trigger Claude within VS Code with a shortcut (default is Cmd+Esc on Mac or Ctrl+Esc on Windows) or by clicking the Claude icon. Claude will also automatically share context about your open file or selection – for example, if you have code highlighted, Claude is aware of it.
- **JetBrains (IntelliJ, PyCharm, WebStorm, etc.):** Install the Claude Code plugin from the JetBrains Marketplace. After installing, restart your IDE. Then, either open the terminal in the IDE and run *claude* once (which connects the CLI to the IDE) or use the plugin's UI to start a Claude chat. The integration will enable Claude to open diffs and file edits directly in the editor rather than just viewing the terminal output. For example, when Claude makes changes, you can have it display a diff in the IDE's diff view for easier review.
- **Verify the integration:** Once installed, try out a quick command to ensure it's working. For VS Code, you might select a piece of code and press Cmd+Esc to ask Claude, "What does this code do?" Claude's answer should appear, demonstrating it knows the context of your selection. For JetBrains, you might use the Claude tool window to ask a question about the currently open file. Also, test that file opens work: e.g. say "open app.js" and see if Claude opens that file (it should pop open in your editor).

Working within the IDE means you get nice conveniences: you can edit code as usual but call on Claude for help without leaving the editor. Claude will inherit

your IDE's context (open files, current branch, etc.) when launched this way. If you ever prefer going back to an external terminal, you can still connect to the IDE session by running the */ide* command in Claude's terminal UI to attach to the editor.

By the end of Day 5, you should be comfortable invoking Claude from your IDE – no more context-switching to a separate terminal if you don't want to. This setup often makes using Claude feel more natural as part of coding, especially when reviewing diffs or navigating code with a GUI.

Day 6: Customizing Claude's Memory

Now that you have some experience, it's time to refine the project memory (CLAUDE.md) that Claude uses. Claude automatically loads any CLAUDE.md file in your project (and even in parent directories) to guide its behavior. Today, you'll edit and improve this file so Claude has the best guidance about your project.

- **Open CLAUDE.md:** You can open it directly in your editor or ask Claude to open it. Review what Claude generated on Day 1. Typically, it includes sections such as standard commands, coding style notes, and key project information, among other relevant details. Now think: What's missing? Are there any conventions or gotchas in your codebase that Claude should be aware of? For example:
    - **Build and test commands**: Ensure the memory lists how to run the project, build it, and run tests. E.g., "npm start launches the dev server on port 3000" or " npm run test runs unit tests". This way, Claude can use these commands without needing to ask each time.
    - **Code style guidelines**: Add bullet points for any style preferences your team follows (indentation, naming conventions, etc.). For instance, "Use async/await for new code, avoid callbacks" or "Follow Airbnb JavaScript style for commas and semicolons". Claude will then adhere to these rules when generating code.
    - **Architectural patterns**: If your project has a specific structure (like "we use MVC" or "all API calls go through apiClient.js "), note that. E.g., "Controllers should not directly access the database – use the Model layer" can prevent an AI suggestion that violates your architecture.
    - **Common pitfalls or warnings**: If there are common mistakes that new developers often make, include them here. E.g., "IMPORTANT:

Every new React component must be added to routes.js or it won't load." – marking it as IMPORTANT can help Claude remember.

- **Edit and organize:** Claude.md is just a Markdown file – feel free to organize it with headings and bullet points for clarity. For example, you might group content under "# Testing" (listing how to run tests or how to generate coverage) or "# Database" (noting migration commands or connection details). Keep entries concise and specific. For instance, "Use 2-space indentation for JSON files" is clear, whereas "format properly" is too vague. Structure helps, too; Claude will parse this each session.
- **Add via Claude or manually:** You can manually edit the file in your IDE. Alternatively, use Claude's built-in shortcuts: typing # at the start of a prompt lets you add a memory entry on the fly. For example, *# Always run \ npm run lint* before committing code – Claude will ask which memory file to save it in (choose project CLAUDE.md). This is handy when you realize mid-session that a piece of info should be persistent. There's also the */memory* command, which opens the memory file for editing.

After updating CLAUDE.md, **save it and commit the changes** (so your team benefits too). Claude Code will use the updated instructions next time. A well-tuned CLAUDE.md means Claude's suggestions will be more on-point for your project's needs. Remember, you can update this file at any time as the project evolves. It's a good idea to revisit it periodically to add new guidelines or remove outdated ones.

Pro Tip: At Anthropic, they even run CLAUDE.md through a prompt improver tool to make instructions more effective. You can emphasize critical rules with words like "MUST" or "NEVER" if you find Claude occasionally not following a guideline. Don't overload the memory with too much, though – keep it relevant and high-impact. The quality of the information here is more important than the quantity.

Day 7: Using Claude for GitHub Workflow 💻

Your project likely uses GitHub for version control and code reviews. Claude can be a big help here! Today, you'll integrate Claude into your GitHub workflow – from crafting commit messages to managing pull requests. Make sure you have the GitHub CLI (gh) installed and authenticated on your machine (Claude can use it if available):

- **Crafting commit messages**: Instead of writing commits manually, let Claude do it. After you've made some changes with Claude's help (for

example, the bug fix from Day 4 or improvements from Day 6), you can simply tell Claude: “Commit these changes with a message”. Claude will analyze the *git diff* and automatically generate a descriptive commit message. It often mentions relevant context and reasons for changes. This saves time and ensures consistency. (Tip: Claude understands instructions like “commit with the message: <text>” or even just “git commit,” where it will prompt itself to compose a message.)

- **Pull request creation**: If you’re on a new branch and ready to open a PR, ask Claude to do it. For example: “Open a pull request titled ‘Fix profile null pointer bug’”. Claude knows that the shorthand “PR” means a GitHub Pull Request and will use the GitHub CLI or API to create it, along with a proper title and description. It can include a diff summary in the description. This eliminates the need for fiddling with the GitHub web UI or remembering CLI syntax.
- **Reviewing and addressing feedback**: Claude can assist in responding to code reviews. If your PR gets review comments, you can copy them or have Claude fetch them (Claude can use *gh* to read comments if permitted). Then, ask Claude to help fix the issues. For instance, “Apply the reviewer’s suggestion to rename the *userId* variable and update the docs accordingly.” Claude will make the changes and even draft a reply. It’s even capable of handling multiple comments at once: “Fix all PR review comments” will prompt Claude to review each and adjust the code (just be sure to double-check each change).
- **Advanced Git queries**: You can use Claude to search git history or triage issues. Try something like, “What’s the history of changes to OrderService.js ?” Claude can call *git log -p* or similar to summarize changes (including who made them and when.) Or, “List all open GitHub issues labeled bug” – Claude could use the GitHub API or CLI to fetch and summarize them (if you allow internet or API access). This is a glimpse of using Claude as an assistant for project management tasks as well.
- **Safety and permissions**: By default, Claude will ask permission before performing potentially destructive Git operations (like a commit or push). You can grant permission when prompted (and even set it to “Always allow” for that operation if you’re comfortable.) For example, when Claude attempts *git commit*, you’ll confirm it. If you find the prompts too frequent, you can adjust the allowlist via /permissions (e.g., always allow Bash(git commit:*) to let all commits go through without confirmation.) Use this power carefully – perhaps allow auto-commit on a branch, but you might not want to always auto-push without review.

By the end of Day 7, you should have seen how Claude can take over routine

GitHub tasks. Many Anthropic engineers reportedly use Claude for 90% of their git interactions – that includes writing commit messages, creating PRs, and even resolving merge conflicts with Claude's help. It's quite freeing to delegate these tasks to the AI, allowing you to focus more on coding logic.

Day 8: Tackling a Jira Ticket with Claude (Manual Approach) 📋

Now, it's time to involve your project management system. If your team uses Jira (or a similar ticket tracker), Claude can assist in transitioning from ticket to code. Today, you'll practice using Claude to work on a real Jira issue, providing the context manually.

- **Select a Jira ticket**: Choose a ticket or user story assigned to you (preferably a medium-sized task, not a massive feature). Open the ticket in Jira and copy the relevant details, including the title, description, acceptance criteria, and any relevant discussions. Essentially, gather all the information that a developer would need to understand the task.
- **Provide context to Claude**: In your Claude session (IDE or terminal), paste in the ticket details. You might say: "Here's a new task: [PASTE Jira description]. Given this, let's plan the implementation." This is important because Claude itself doesn't have direct access to your Jira (yet – integration comes tomorrow). By pasting the info, you ensure Claude fully understands the requirements and scope.
    - **PRO TIP**: Hit shift+tab+tab to enter into planning mode so Claude spends extra time understanding the problem and coming up with a solution.
- **Analyze and plan**: Ask Claude to summarize or outline a solution. For example: "Summarize the problem and propose a step-by-step plan to implement this feature." Claude will digest the Jira text and hopefully produce a coherent plan. It might list which parts of the codebase require changes, what functions to add, and so on. Review this plan. This is where you bring your knowledge – if Claude's plan misses something (maybe a non-obvious business rule mentioned in a meeting, or a technical constraint not in Jira), correct it. You can say, "That looks good, but ensure we also handle the case where the user is an admin (as per an email discussion). Update the plan for that." Claude will incorporate that detail.
- **Implement step by step**: With an agreed plan, have Claude implement it.

You can go one step at a time or try it all at once:
    - "Start by creating the new API endpoint file and stub out the handler." – Claude will create the file and basic code.
    - "Now implement the business logic in that handler, following the acceptance criteria." – Claude writes the code.
    - "Add unit tests for the new functionality as described in the ticket." – it can generate tests (remember to specify key scenarios from the acceptance criteria.)
    - "Run the tests." – Allow Claude to execute them and see if they pass.
    - "If any tests failed, fix the code accordingly." – iterate until green.
- **Validate acceptance criteria**: Double-check that what's implemented truly meets the ticket. You can even ask Claude to go through each acceptance point: "Verify that the code covers each acceptance criterion (A, B, C) and explain how." This can surface any missed pieces. If something is not addressed, have Claude add it.

At the end of Day 8, you should have a completed feature or fix that aligns with a Jira ticket, accomplished with Claude's heavy involvement. You likely pasted quite a bit of text for context – that's fine for now. The goal was to simulate how Claude can be used on real work items by feeding it the necessary information. This manual context approach is useful anytime an AI doesn't have direct access to a requirement – simply paste the requirement in.

(Note: If the pasted ticket text was very long, consider using Claude's ability to read files: you could save the description to a local text file and use "read ticket.txt" to avoid hitting token limits or messing up formatting. Claude can also summarize long text if needed.)

Day 9: Automated Jira Integration & Puppeteer – Going Full Agent

Today we'll level up: connecting Claude directly to Jira and even giving it the ability to interact with a browser via Puppeteer. This moves toward a world where Claude can autonomously fetch task details and test the application like a human would. Only attempt this if you're comfortable with the earlier steps, as it involves configuration.

1. Connect Claude to Jira (MCP Integration): Anthropic and Atlassian have partnered on a Remote MCP Server for Jira/Confluence. This means Claude can use Jira's API through a secure channel. Here's how to set it up: - Obtain credentials: Make sure you have access to your Jira Cloud instance and an API token, or be ready to do an OAuth login. If your admin has enabled the Atlassian integration, you might not need a manual token. Otherwise, you can create a personal API token through your Atlassian account settings.

   - Add the Atlassian MCP server: Use Claude's CLI to add the integration. For example, run:

   claude mcp add atlassian --transport sse https://mcp.atlassian.com/v1/sse

   This registers the Atlassian remote MCP endpoint (which is hosted by Atlassian) with Claude Code. Claude can now communicate with Jira/Confluence through this channel.

   - Authenticate: After adding, you may need to authenticate the connection. Run the command /mcp inside Claude to view the configured servers and follow any authentication prompts. For Atlassian's server, a browser window might open asking you to allow access (OAuth 2.0). Approve it so Claude can act on your behalf within Jira (only within your permissions).

   - Test Jira access: Try something simple to ensure it works. For instance, type: "Summarize Jira issue PROJ-123" (replace with a real project key and issue number). Claude should use the integration to fetch the issue details and then provide a summary of them. If this succeeds, congratulations – Claude is effectively reading from Jira directly! You can also try: "Create a new Jira ticket titled 'Test from Claude' in project PROJ" to see it write to Jira (you can delete it afterward). Claude can now perform tasks such as creating issues, commenting, or transitioning tickets, all through the AI interface, respecting your Jira permissions.
2. Add Puppeteer for Browser Testing: Puppeteer MCP gives Claude a "robot browser" it can control. This is powerful for frontend-heavy projects or end-to-end testing: - Install Puppeteer MCP: In your project directory, run:

   claude mcp add puppeteer -s project -- npx -y @modelcontextprotocol/server-puppeteer

   This command installs the Puppeteer MCP server (from the

modelcontextprotocol open-source registry) and registers it at project scope. The next time you launch Claude in this project, it can use Puppeteer. (The -s project flag means it will add an entry to a .mcp.json in your repo so teammates can also get this config.)

- Allow Puppeteer actions: The first time Claude tries to use the browser, it may ask for permission. Specifically, you might get prompts like “Allow puppeteer_navigate?” or “Allow puppeteer_screenshot?”. You can approve each, or preemptively update your permissions (using /permissions or editing .claude/settings.json) to always allow the Puppeteer MCP’s actions . For now, granting as needed is fine.

- Use Puppeteer in a task: Let’s revisit the Day 8 ticket, or choose a new one that involves UI changes. With the app running locally (make sure Claude knows how to start it – e.g. if npm start launches your dev server, that command should be in CLAUDE.md or you can just instruct Claude to run it), you can have Claude visually test the application.

For example:

- “Launch the application and navigate to the user profile page in a browser.” – Claude will likely run the dev server (if told how) and then use Puppeteer to open the page.

- “Take a screenshot of the profile page after the changes.” – Claude can use Puppeteer to take a snapshot of the page. It will save an image (often in a temp path) and might display a reference or even show it if your interface supports image output. This allows you to verify that the UI appears correctly.

- “Simulate a user updating their profile with name ‘Alice’ and submit – verify the result.” – Claude can script form interactions via Puppeteer (typing into fields, clicking buttons) and then observe what happens (e.g., check if a success message appears, or take another screenshot). Essentially, Claude can now perform an end-to-end test: it writes code, starts the app, opens a headless browser, interacts with the UI, and confirms the behavior. This is like having a QA engineer in the loop, but automated. It’s extremely useful for catching UI/UX issues or ensuring that backend changes actually surface correctly in the frontend.

- Iterate with visual feedback: If you have design mocks or expected

screenshots, you can even give those to Claude. For example, provide an image (via a path or link) and ask Claude to match the implementation to the mock. Claude will then repeatedly adjust the frontend code and use Puppeteer to compare the screenshot of your app to the provided mock, refining it until it's close. This is an advanced workflow, but shows how powerful these tools are – Claude can literally see the UI and improve it.

By the end of Day 9, you've essentially unlocked "*agentic coding*" — Claude can autonomously gather requirements (Jira) and verify outcomes (browser tests). Of course, you still oversee the process: you trigger the actions and review the results. But a lot of the heavy lifting of context gathering and result checking can be offloaded. This can significantly accelerate the development of complex tasks.

Reminder: When running such autonomous flows, keep an eye on what Claude is doing. The --mcp- debug flag is useful if something isn't working, as it will log details of the MCP calls . And never forget to update your CLAUDE.md with any new commands (like how to run the app) that Claude might need to recall. If the AI knows exactly how to run and test your project from memory, it can do all these steps with minimal guidance.

Day 10: Reflection and Next Steps 📈

Congratulations on integrating Claude Code into your workflow for two weeks! Today, step back and reflect on how your relationship with the codebase has changed:

- **Productivity and Speed**: Think about the tasks that felt faster with Claude. Did routine coding go quicker (e.g., writing boilerplate or following spec to code)? How about debugging – was pinpointing issues easier? Note specific instances where you saved time or where Claude's assistance was particularly effective.
- **Code Quality and Consistency**: Review the code written with Claude's help. Is it up to your team's standards? Often, Claude can enforce consistency (especially after Day 6's memory tuning) – maybe your code style is now more uniform. On the other hand, identify any bad suggestions you had to correct; this is useful feedback to improve prompts or memory notes.
- **Learning and Understanding**: Did using Claude help you learn parts of the codebase faster? Many developers find that asking Claude questions

is like having a knowledgeable teammate, which can accelerate onboarding to unfamiliar areas. If you felt this, consider how you might use Claude for knowledge sharing among the team (for example, new hires could use the Q&A approach to get up to speed).

- **Collaboration and Workflow Changes**: Reflect on how integrating Claude has impacted your day-to-day. Are you spending less time searching documentation or Stack Overflow? Do code reviews go faster because commit messages and PR descriptions are well-written by Claude? Also, consider your team – if multiple engineers use Claude, you might notice fewer repetitive questions in team chats, as people can self-serve with the AI.
- **Trust and Verification**: It's important to assess how much you trust Claude now versus Day 1. Initially, you may have been skeptical and double-checked everything. Over time, perhaps you gained confidence in letting it handle more tasks (such as committing code or running tests). It's healthy to keep some skepticism – AI can make mistakes. But ideally, you now have a sense of when to rely on Claude and when to be extra careful. Write down any guidelines you've developed for yourself in using the AI (e.g., "Always review code suggestions for security issues" or "Use think harder for complex logic to get better reasoning from Claude").

Finally, consider next steps. How will you continue using Claude? Perhaps make a plan to keep CLAUDE.md updated (maybe assign someone each sprint to update it with new learnings). You might also explore more advanced Claude features or upcoming integrations. For example, Anthropic is expanding Claude's integration ecosystem (Confluence documentation, other SaaS tools, etc., as hinted at by Atlassian's roadmap). Maybe you'll experiment with those in the future.

Take this day (or your next retrospective) to discuss with your team as well. Sharing experiences can uncover valuable tips (such as useful prompt phrasings that someone has discovered) or address concerns (perhaps someone has noticed Claude struggling with a specific framework, which could lead to a memory update or a bug report to Anthropic). The end goal is not just a one-time productivity boost, but an ongoing partnership with AI in your development cycle.

Hopefully, you now feel comfortable with Claude as a powerful assistant rather than a novelty. You've set it up, tuned it to your project, and used it in various scenarios from coding to testing to documentation. As you continue, refine the process further. Much like a human team member, Claude will get better with good guidance and as it "learns" your project conventions. Happy coding with

your new AI pair programmer!